Математика глубокого обучения: введение

Автор: Леонид Берлянд, Пьер-Эммануэль Жабен

Тип файла: pdf

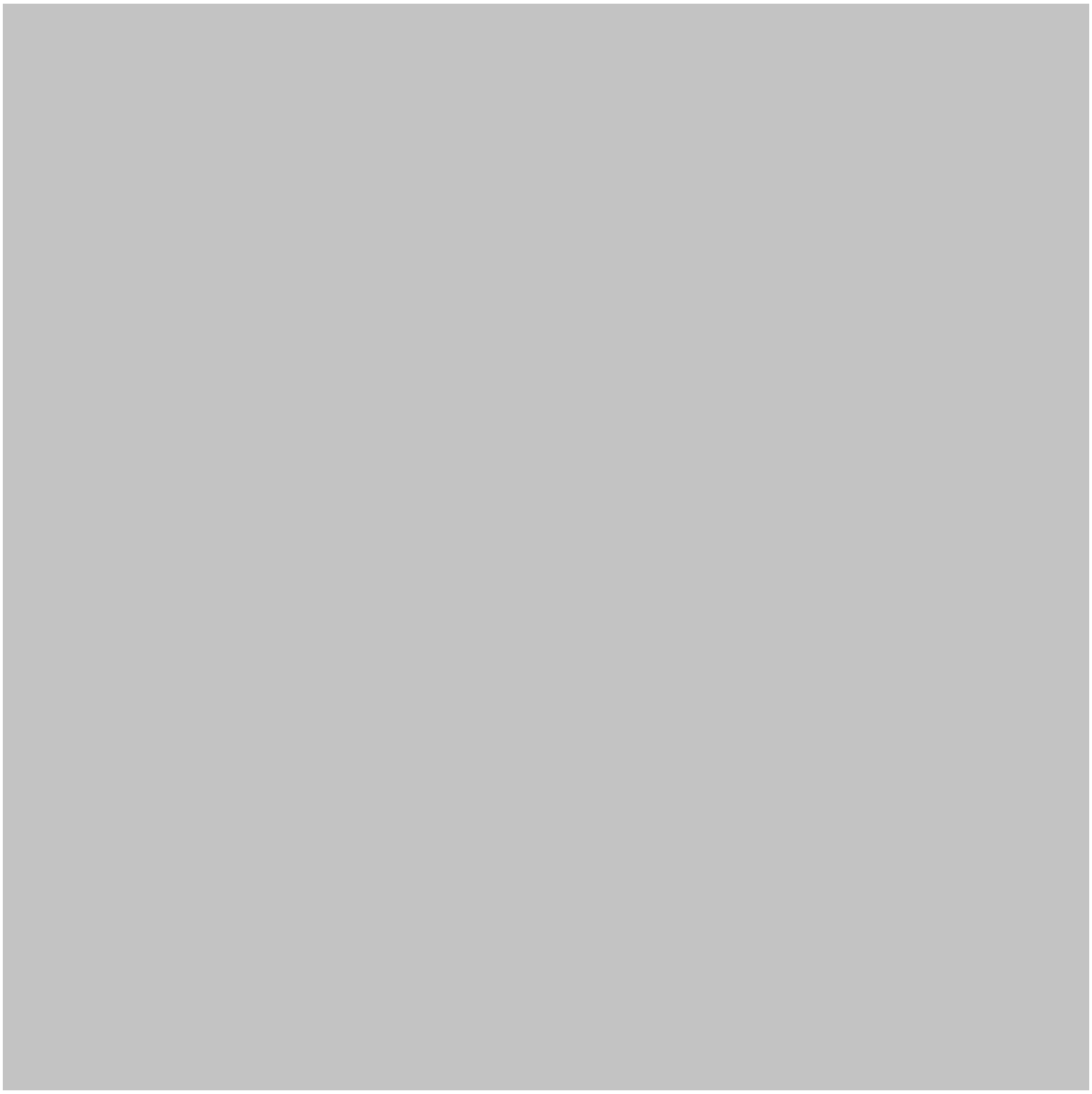
Размер: 11,5 МБ

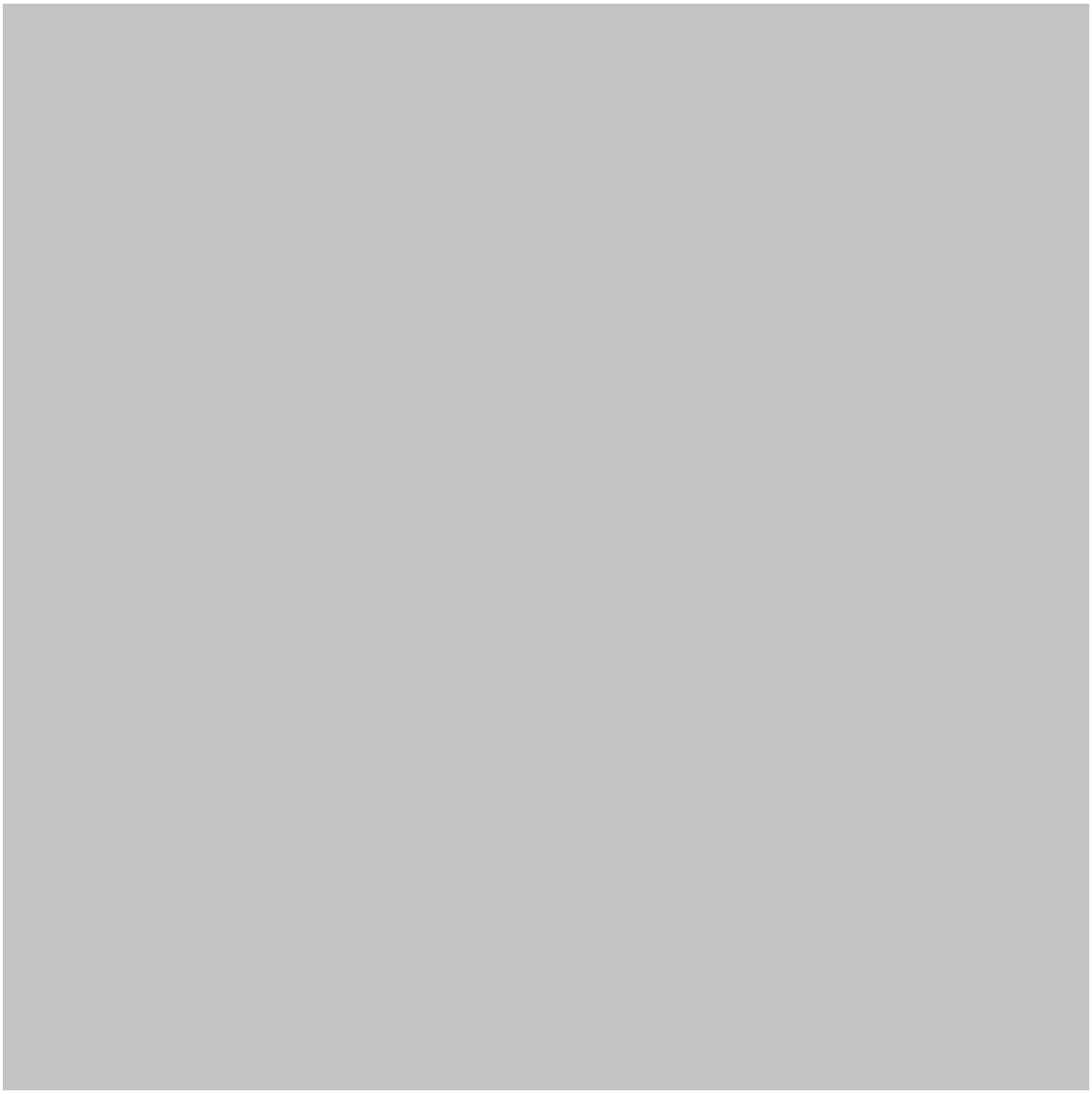
Язык: Английский

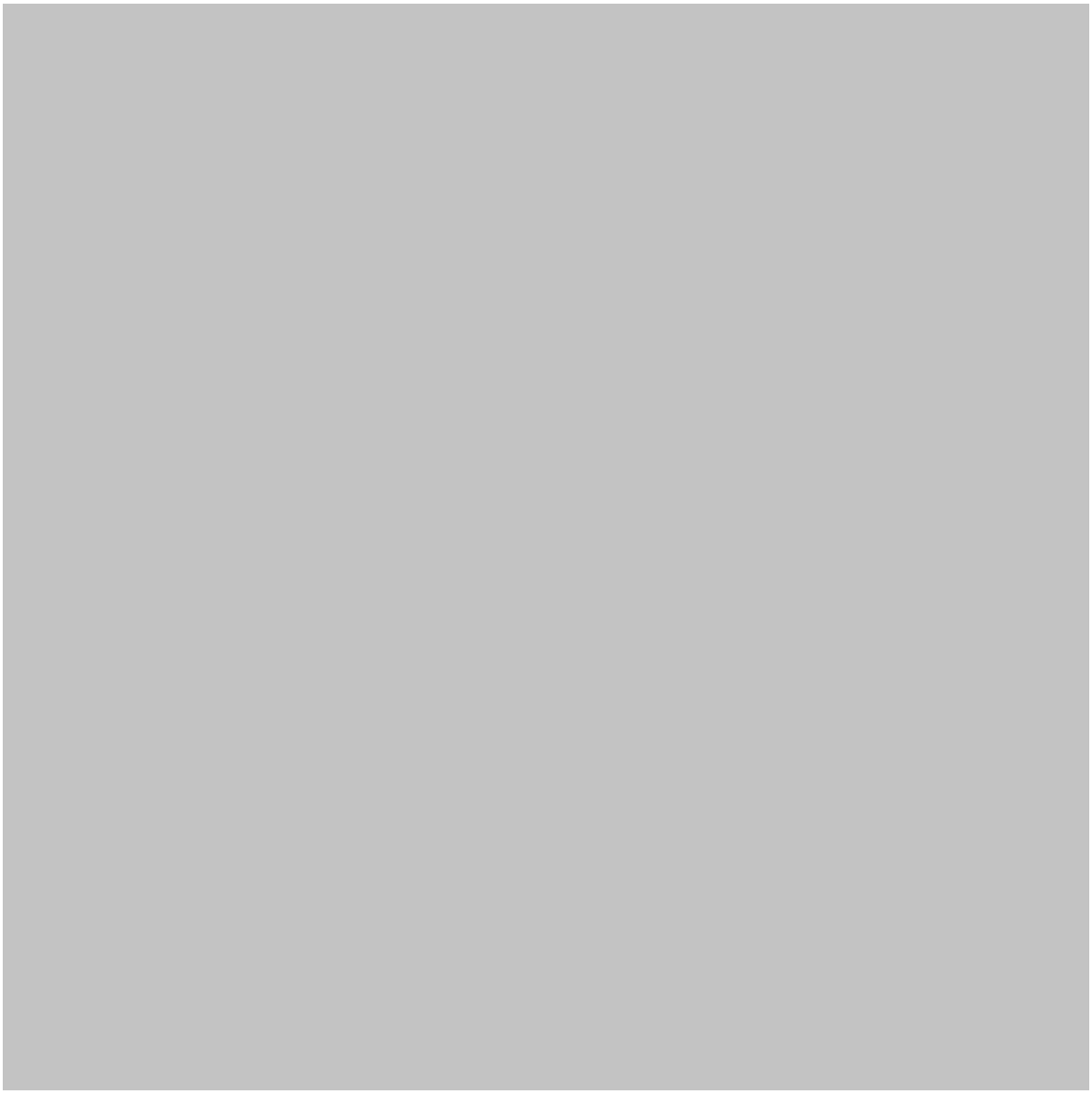
Страниц: 132

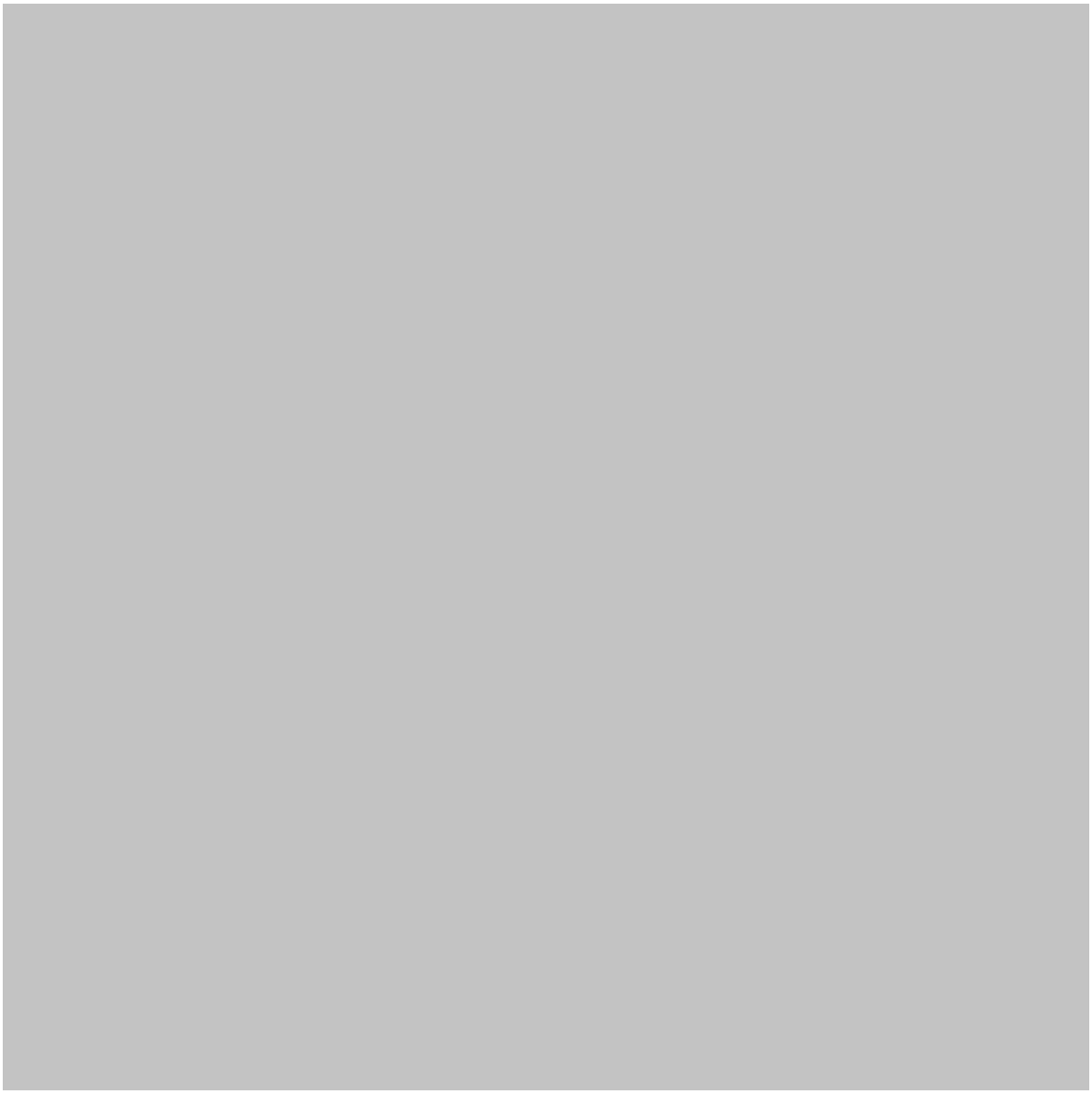
Математика глубокого обучения: введение

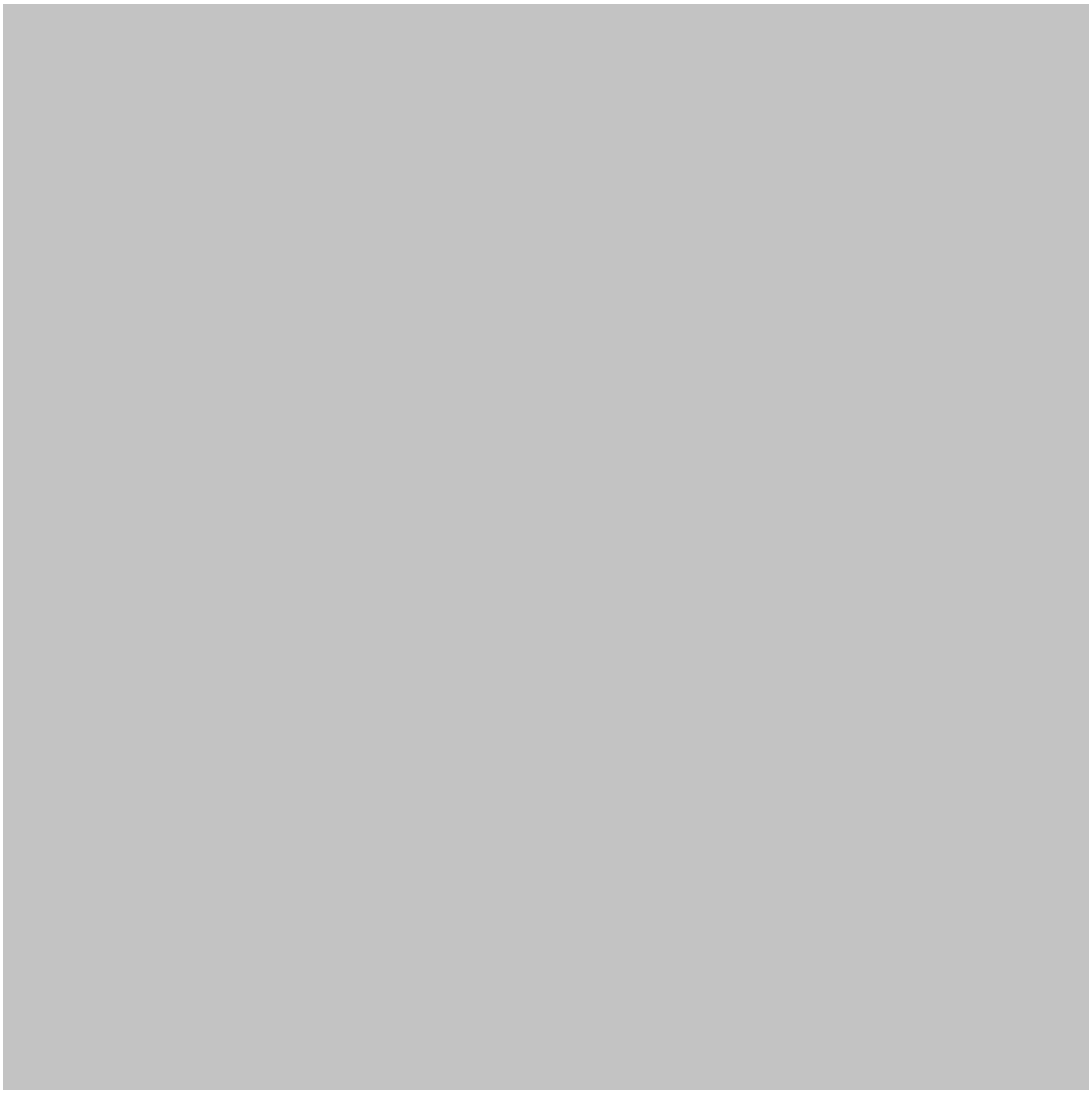
Глубокое обучение может показаться набором сложных алгоритмов, но в основе современных нейронных сетей лежит удивительно структурированная математическая система. Линейная алгебра представляет данные, математический анализ описывает процесс обучения моделей, теория вероятностей помогает справляться с неопределенностью, а оптимизация обеспечивает механизм для улучшения прогнозов. Вместе эти математические идеи позволяют инженерам создавать системы, которые распознают изображения, понимают язык, анализируют показания датчиков, выявляют аномалии и делают прогнозы на основе огромных массивов данных.











Для студентов инженерных специальностей и профессионалов понимание этих основ ценно тем, что оно меняет представление о системах глубокого обучения. Вместо того чтобы воспринимать нейронную сеть как загадочный черный ящик, можно использовать математические концепции, чтобы понять, что происходит внутри каждого слоя и почему обучение работает.

В этой статье мы расскажем об основах математики, лежащих в основе глубокого обучения, не требуя от читателя глубоких познаний в математике. Наша цель — связать абстрактные концепции с практическими инженерными примерами. 🚀

Теория предыстории

Почему математика важна для глубокого обучения

Нейронная сеть получает числовую информацию и преобразует ее с помощью последовательности математических операций.

Изображение может быть представлено в виде числовых значений пикселей. Датчик температуры выдает числовые значения. Система обработки текста преобразует слова в числовые представления.

Затем нейронная сеть многократно выполняет такие операции, как:

- Векторные преобразования
- Умножение матриц
- Сложение
- Нелинейные преобразования
- Вычисления вероятностей
- Вычисления градиента
- Оптимизация

Таким образом, математика предоставляет язык, с помощью которого можно описать, как информация проходит через модель.

Четыре математических столпа

Математику глубокого обучения можно условно разделить на четыре области:

Mathematical area	Main purpose in deep learning
Linear algebra	Represents and transforms data
Calculus	Determines how model parameters should change
Probability	Represents uncertainty and predictions
Optimization	Finds parameter values that reduce prediction error

Statistics also plays an important role in evaluating models and understanding datasets.

From data to prediction

Consider an industrial vision system designed to identify defective components.

A camera captures an image → numerical pixels represent the image → neural-network layers transform those numbers → the final layer produces prediction values → the system identifies the likely class.

At every stage, mathematical operations control the transformation.

Definition

What is the mathematics of deep learning?

The **mathematics of deep learning** refers to the collection of mathematical concepts and techniques used to represent data, construct neural networks, train models, measure errors, and optimize model parameters.

A simplified deep-learning pipeline looks like this:

Data → Representation → Neural Network → Loss → Gradients → Optimization → Updated Model

The process repeats many times during training.

What is a neural network mathematically?

A neural network can be viewed as a sequence of mathematical transformations.

Each layer generally receives numerical inputs, applies parameters such as weights and biases, performs a transformation, and passes the result to another layer.

The presence of nonlinear activation functions allows networks to represent complicated relationships rather than only simple linear relationships.

Why deep networks need nonlinear functions

Imagine trying to model a complicated relationship between thousands of sensor measurements and machine failure.

A purely linear transformation has significant limitations. By introducing nonlinear functions between layers, a deep network can construct increasingly sophisticated representations.

Common activation functions include:

- ReLU
- Sigmoid
- Tanh
- Softmax

- GELU

Each has different mathematical and computational characteristics.

Step-by-Step Explanation

Step 1: Represent information numerically

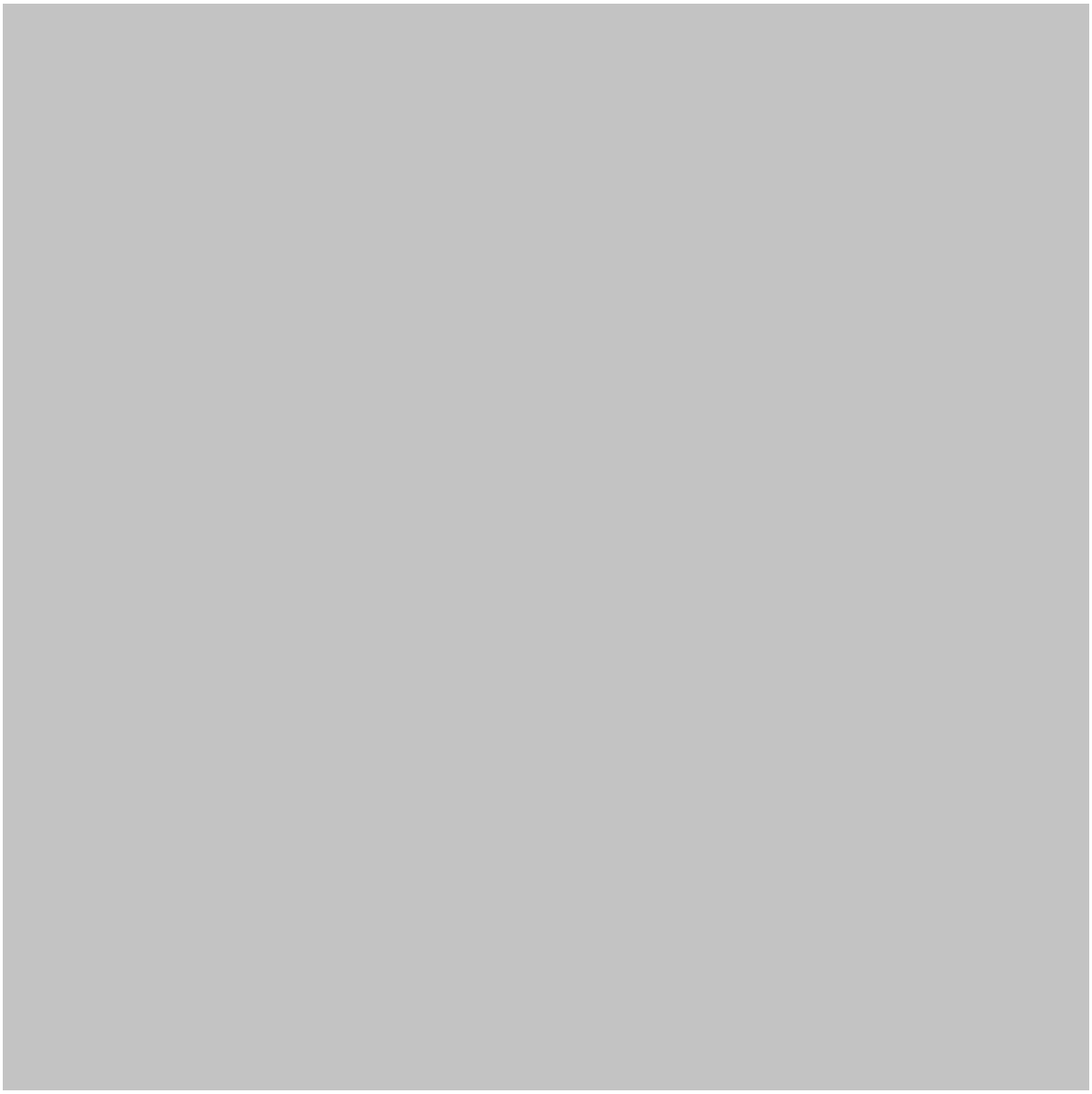
The first requirement is numerical representation.

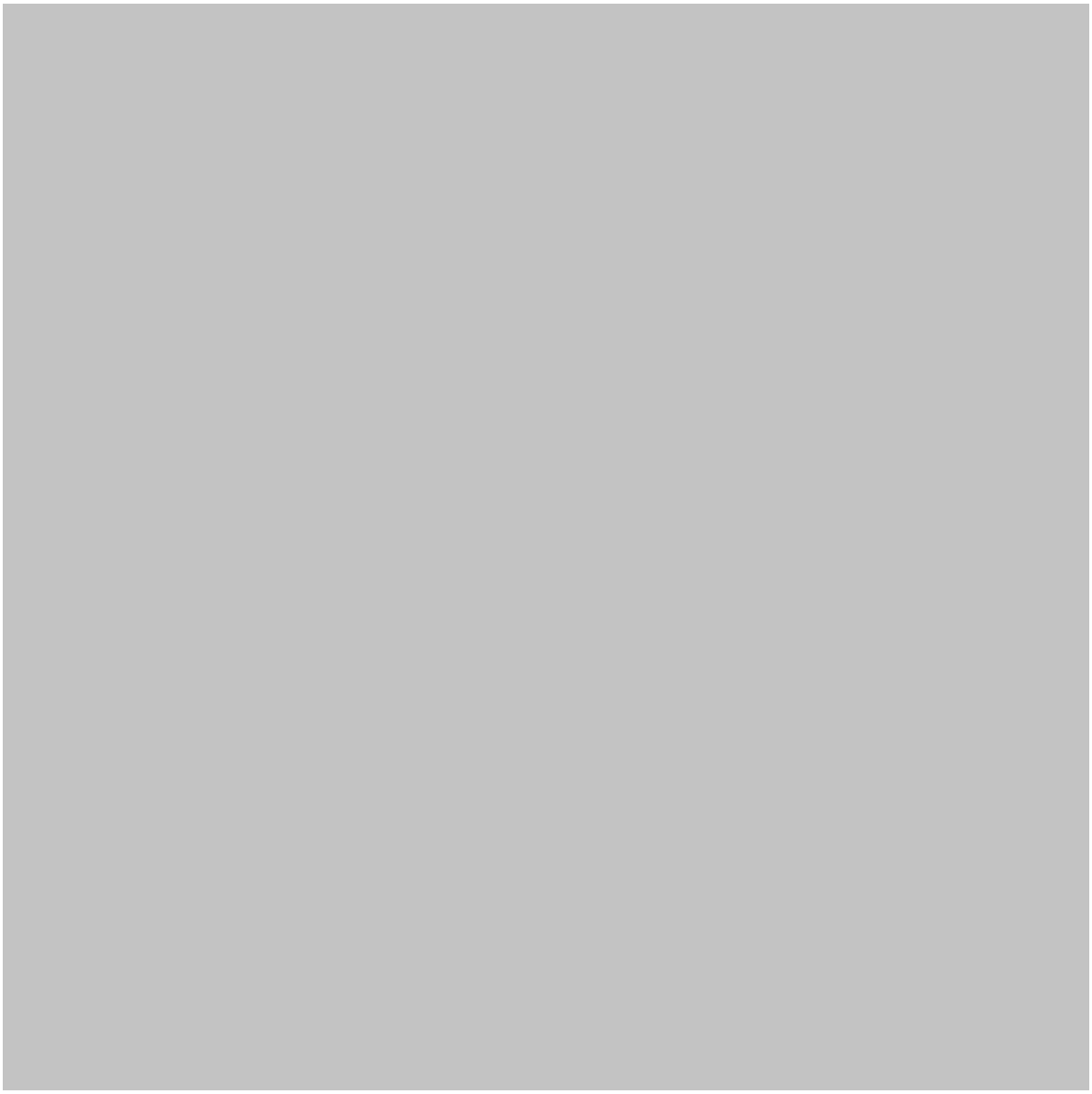
A simple engineering dataset might contain:

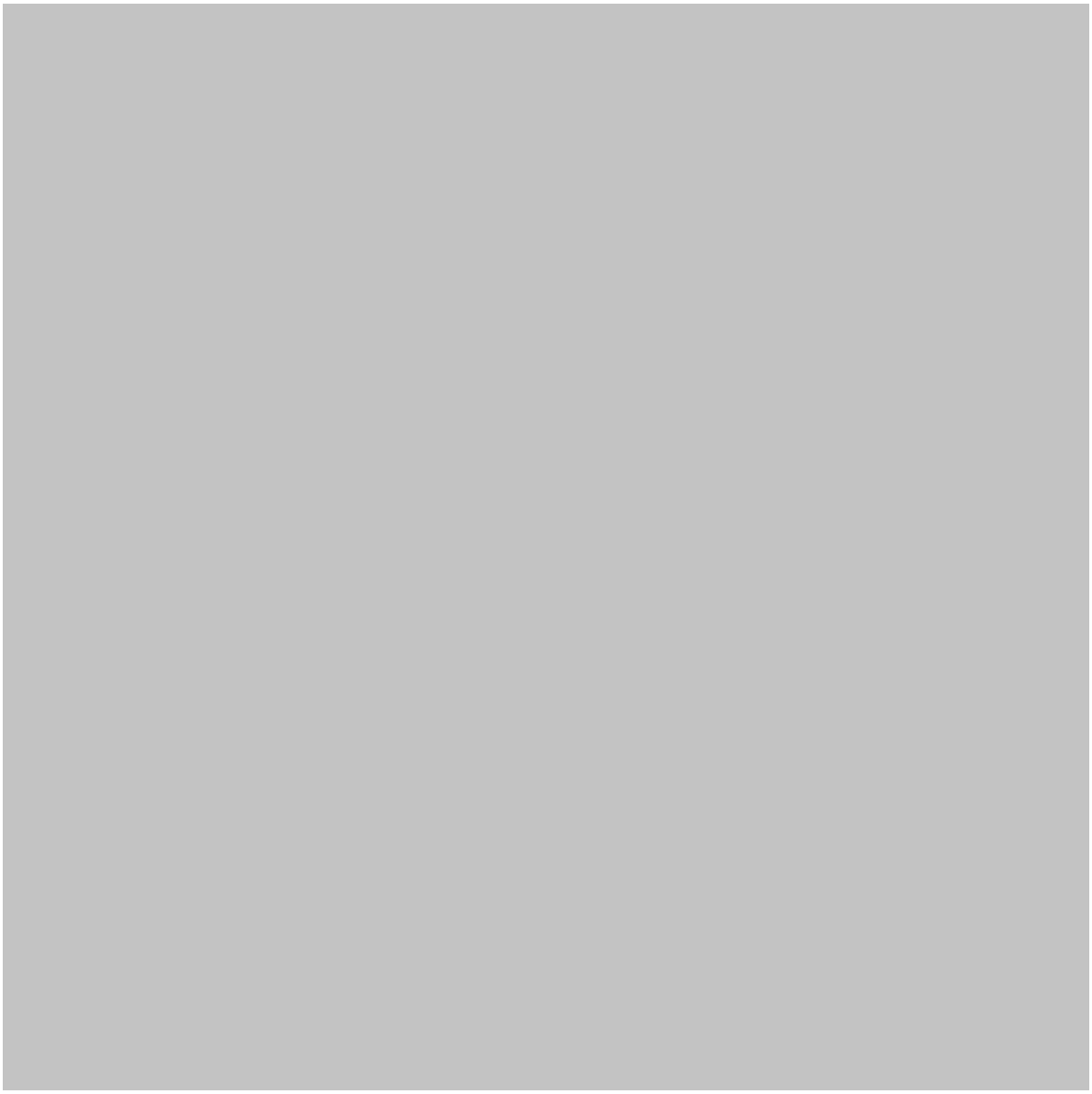
- Temperature
- Pressure
- Vibration
- Rotation speed
- Electrical current

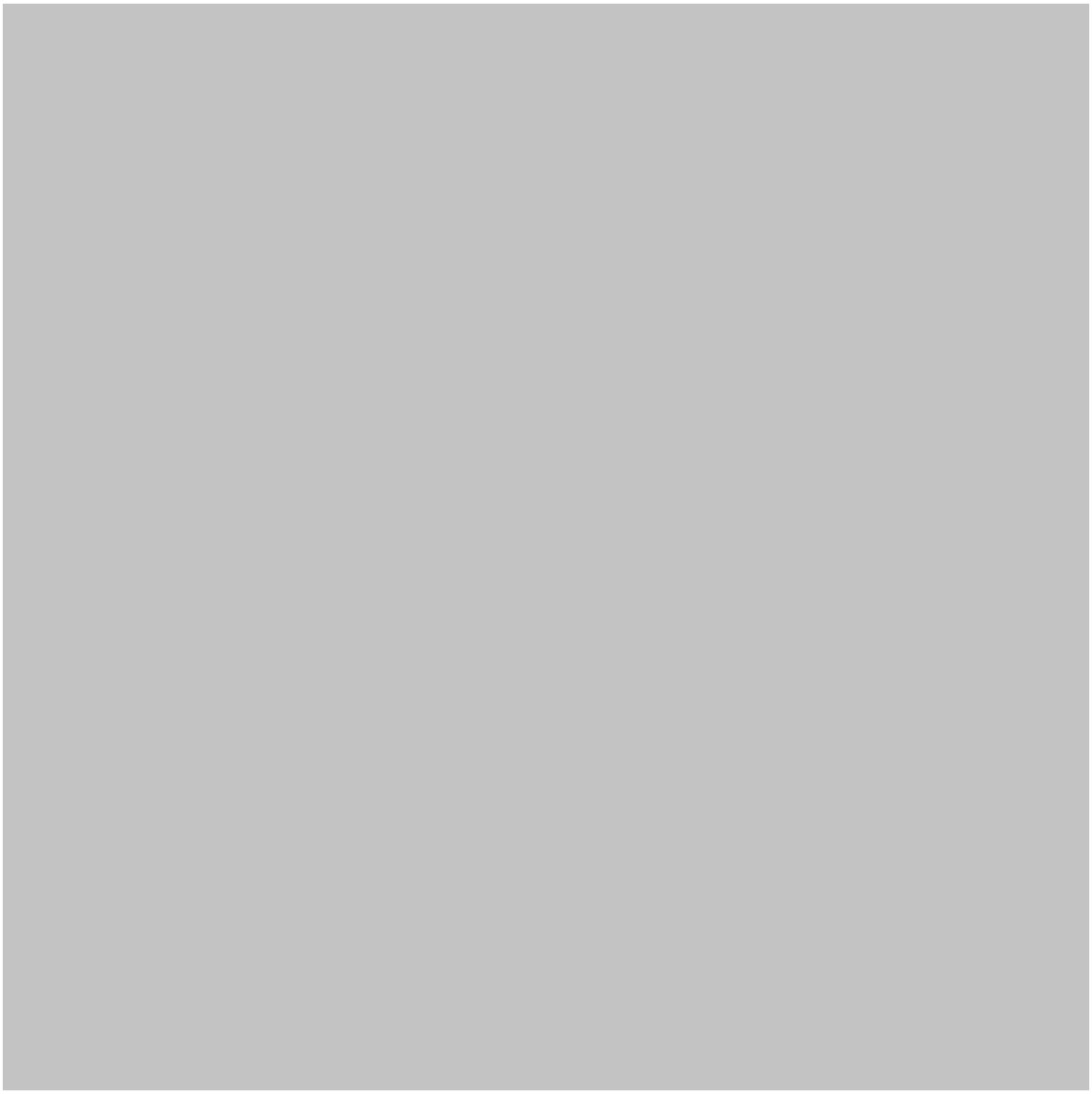
These measurements can be organized into vectors or matrices.

For an image, a matrix can represent pixel intensity values. A color image may require multiple numerical channels.









Step 2: Transform the input

A neural-network layer transforms its incoming numerical representation.

The transformation is controlled by learned parameters.

During training, these parameters gradually change so that the model's predictions become more useful.

Step 3: Apply an activation function

After a transformation, an activation function introduces nonlinear behavior.

ReLU, for example, is popular because it is computationally simple and often helps deep networks train effectively.

Step 4: Repeat through multiple layers

The output of one layer becomes the input of another.

Early layers may learn relatively simple patterns, while deeper layers can combine these representations into increasingly complex features.

In computer vision, this can mean moving conceptually from edges and textures toward shapes and object-level characteristics.

Step 5: Produce an output

The final layer converts the learned representation into a prediction.

Depending on the application, the output might represent:

- A class
- A probability distribution
- A numerical prediction
- A generated sequence
- A segmentation map

Step 6: Measure the error

The model needs a way to determine whether its prediction is close to the desired result.

This is the role of the **loss function**.

Common examples include:

- Mean squared error
- Cross-entropy loss
- Binary cross-entropy
- Huber loss

The loss converts prediction quality into a numerical signal that can guide training.

Step 7: Calculate gradients

Calculus becomes especially important here.

A gradient describes how a function changes when its inputs change.

In deep learning, gradients indicate how changes in model parameters affect the loss.

This information allows the training algorithm to determine which parameters should be adjusted.

Step 8: Update the parameters

An optimization algorithm uses gradient information to modify the model.

Popular optimization methods include:

- Stochastic Gradient Descent
- Momentum-based optimization
- Adam
- AdamW

The cycle then repeats:

Prediction → Loss → Gradient → Parameter update → Prediction

This process may occur millions or billions of times during large-scale training. ⚙

Comparison

Linear models versus deep neural networks

Characteristic	Linear model	Deep neural network
Mathematical structure	Relatively simple	Multiple transformations
Nonlinearity	Limited	Extensive
Feature learning	Usually limited	Often substantial
Parameter count	Often smaller	Can be extremely large
Training requirements	Usually modest	Potentially very high
Interpretability	Often easier	Often more difficult
Suitable problems	Simple relationships	Complex patterns

Traditional programming versus deep learning

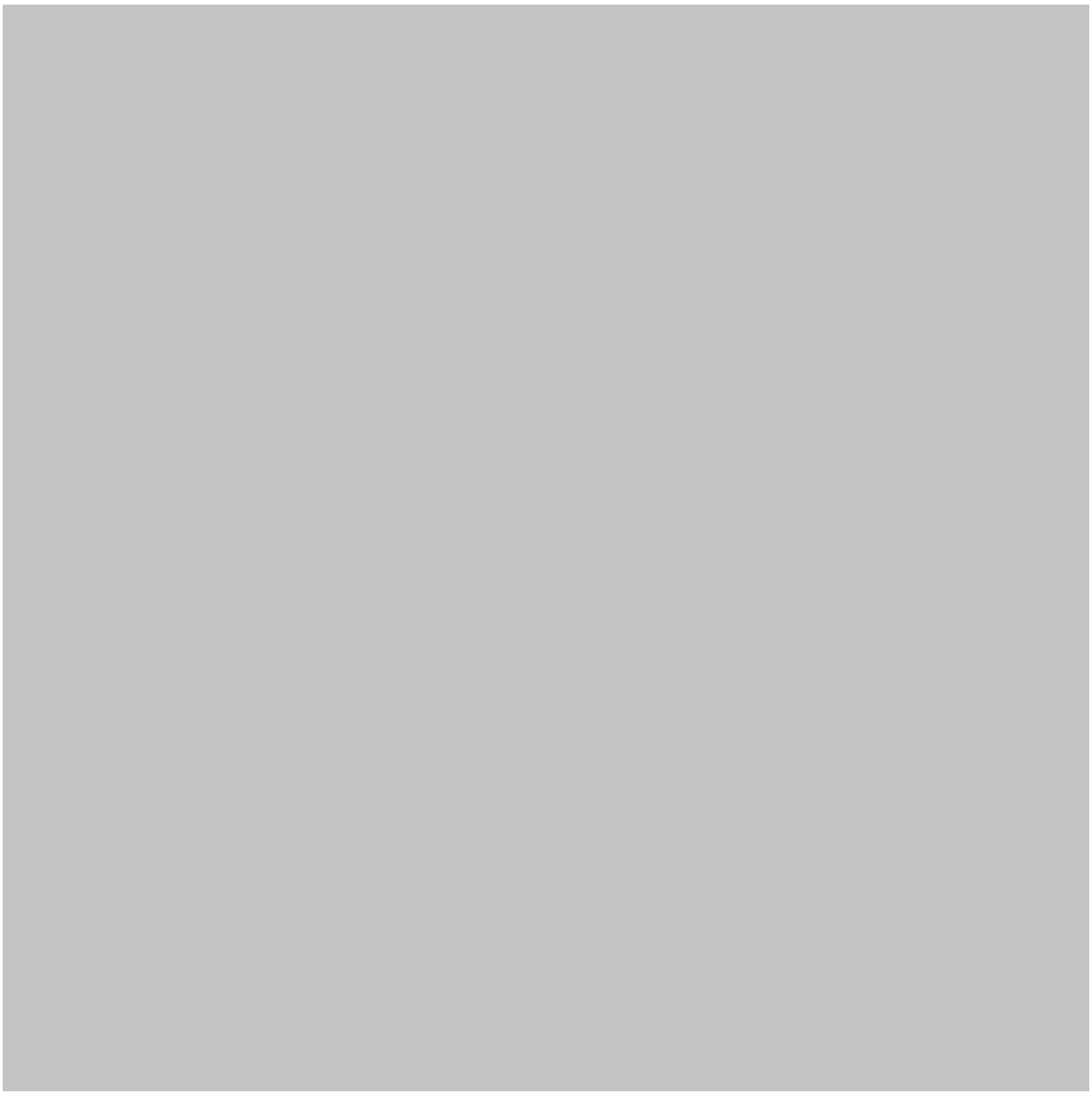
Traditional engineering software generally specifies rules explicitly.

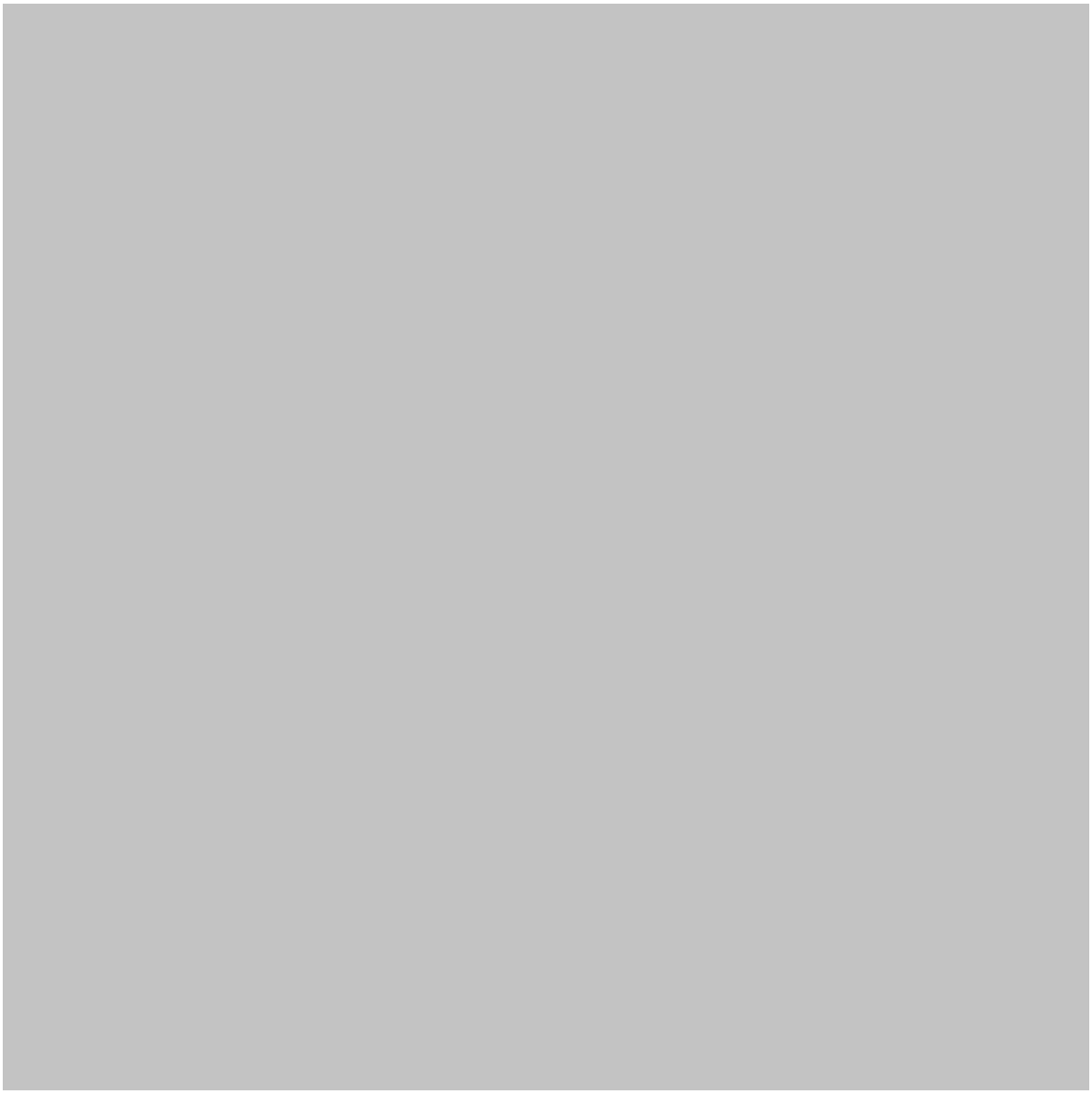
Deep learning instead allows the model to discover useful parameter configurations from examples.

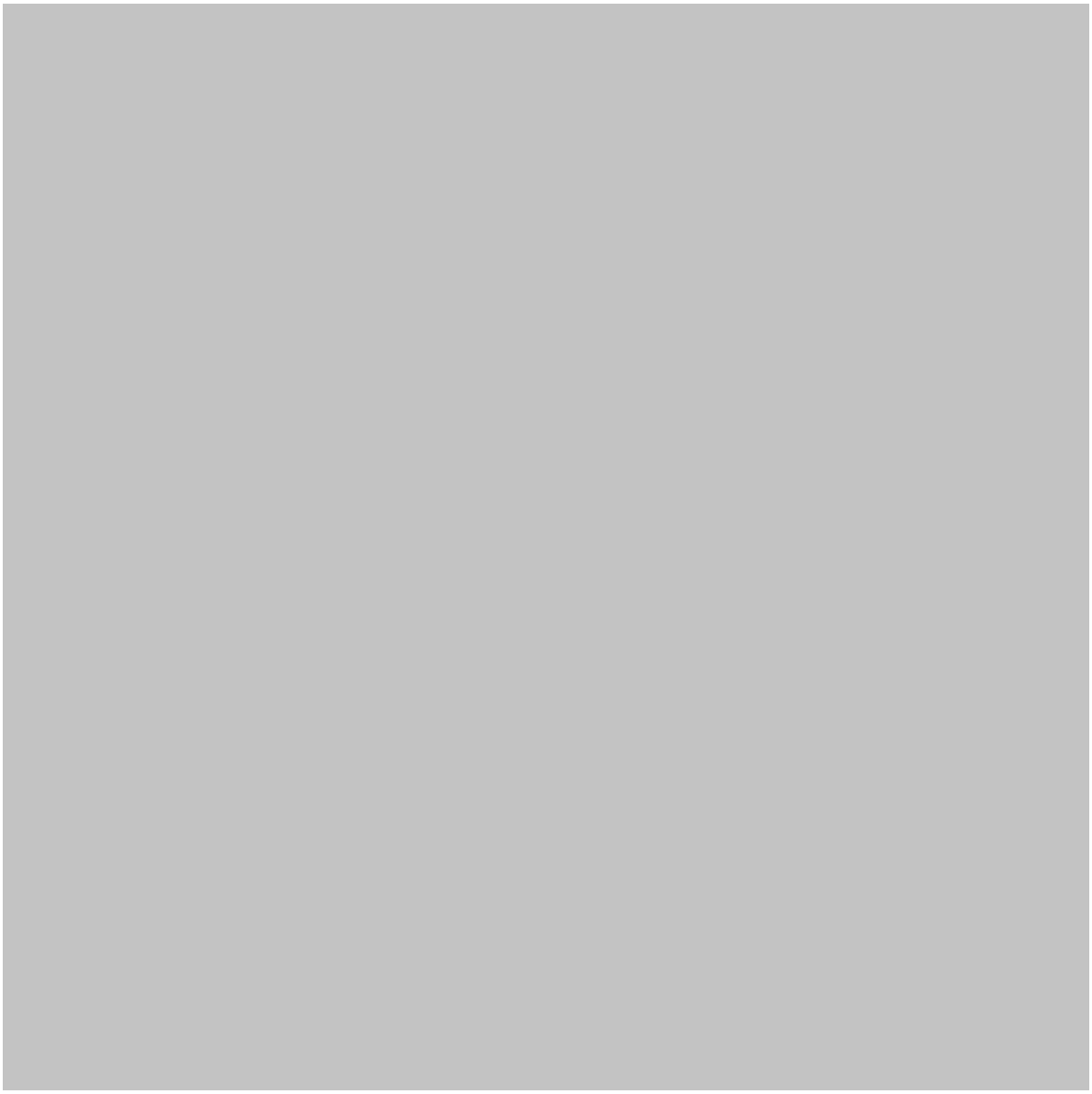
This does not eliminate mathematics. It makes mathematical optimization a central component of the system.

Diagrams & Tables

The mathematical learning loop







A conceptual learning loop can be represented as:

Input Data
↓
Linear Transformation
↓
Activation Function
↓
Neural Network Layers
↓
Prediction
↓
Loss Function
↓
Gradient Calculation
↓
Optimizer
↓

Updated Parameters

↺

Mathematical concept map

Concept	Deep-learning role	Engineering interpretation
Vector	Data representation	Measurements or features
Matrix	Structured data/parameters	Images, datasets, weights
Derivative	Local change	Sensitivity
Gradient	Direction of change	Parameter adjustment information
Probability	Uncertainty	Confidence in predictions
Loss	Prediction error	Model performance signal
Optimization	Parameter search	Improving the model
Statistics	Data analysis	Understanding model behavior

Examples

Example 1: Predictive maintenance

Imagine a manufacturing plant monitoring an electric motor.

Sensors continuously measure vibration, temperature, current, and rotational behavior.

A deep-learning system can learn patterns associated with normal operation and patterns preceding failures.

The mathematics allows the system to transform the sensor information and adjust its internal parameters based on historical examples.

Example 2: Engineering image inspection

A production line uses cameras to inspect manufactured components.

Instead of programming every possible visual defect manually, a neural network can learn from labeled examples.

The mathematical layers progressively transform the image representation until the final system produces a classification.

Example 3: Structural monitoring

Sensors installed on a bridge can collect vibration and environmental information.

A machine-learning system can analyze historical measurements and identify unusual patterns that may warrant engineering investigation.

The system does not replace structural engineering judgment. Instead, it provides an additional analytical tool.

Example 4: Energy forecasting

An energy-management system can use historical demand, weather-related variables, calendar information, and other measurements.

A deep-learning model can learn complex relationships among these variables and generate future demand estimates.

Real-World Application

Computer vision

Deep learning is extensively used for image classification, object detection, segmentation, and industrial inspection.

The mathematical foundation allows image information to be converted into representations that neural networks can process.

Robotics

Robotic systems may combine cameras, force sensors, position measurements, and other data sources.

Deep-learning models can help robots interpret environments and make predictions from complex sensor information.

Aerospace engineering

Machine learning can assist with tasks involving sensor analysis, anomaly detection, predictive maintenance, and simulation-related workflows.

Because aerospace systems are safety-critical, mathematical models generally operate within carefully engineered validation and verification processes.

Civil engineering

Potential applications include:

- Structural health monitoring
- Construction-site image analysis
- Traffic prediction
- Infrastructure inspection
- Material-property prediction
- Project-risk analysis

Mechanical engineering

Deep learning can support:

- Fault detection
- Predictive maintenance
- Manufacturing inspection
- Process optimization
- Design exploration
- Sensor-data analysis

Electrical engineering

Applications include:

- Load forecasting
- Signal classification
- Equipment monitoring
- Fault detection
- Power-quality analysis
- Renewable-energy forecasting

Common Mistakes

Treating deep learning as only a coding problem

A programming framework can build a neural network quickly, but understanding why it behaves as it does requires mathematical foundations.

Ignoring data representation

Poor numerical representation can limit model performance regardless of network architecture.

Confusing correlation with causation

A neural network can identify statistical relationships without establishing that one variable causes another.

This distinction is particularly important in engineering decision-making.

Using unnecessarily complicated mathematics

Beginners sometimes assume they must master every advanced mathematical topic before starting deep learning.

A more practical approach is to learn mathematics progressively alongside machine-learning concepts.

Ignoring scaling

Features with dramatically different numerical ranges can create optimization difficulties.

Normalization or standardization can often improve training behavior.

Assuming a lower training loss always means a better system

A model can perform extremely well on training data while performing poorly on previously unseen data.

This is related to **overfitting**.

Challenges & Solutions

Challenge: High-dimensional data

Modern engineering datasets may contain thousands or millions of variables.

Solution: Use appropriate representations, dimensionality-reduction techniques when justified, feature selection, and architectures designed for the data type.

Challenge: Vanishing or unstable gradients

Very deep networks can encounter difficulties when gradient signals become extremely small or excessively large.

Solution: Modern architectures, appropriate initialization, normalization techniques, activation functions, and optimizers can help.

Challenge: Computational requirements

Large models can require substantial GPU or accelerator resources.

Solution: Engineers can consider smaller architectures, transfer learning, efficient numerical precision, batching, and hardware acceleration.

Challenge: Limited training data

High-quality labeled engineering data can be expensive.

Solution: Transfer learning, data augmentation, synthetic data, semi-supervised approaches, and careful experimental design may reduce data requirements.

Challenge: Model interpretability

Complex neural networks can be difficult to explain.

Solution: Use appropriate interpretability methods, sensitivity analysis, visualization, controlled experiments, and domain-specific validation.

Case Study

Deep learning for predictive machine maintenance

Consider a factory containing hundreds of rotating machines.

Each machine produces streams of measurements from vibration, temperature, electrical current, and rotational sensors.

Initially, engineers might analyze these measurements using thresholds and manually designed rules.

A deep-learning project could follow this workflow:

Data collection → Cleaning → Feature representation → Model training → Validation → Deployment → Continuous monitoring

Historical sensor records can be associated with maintenance events.

The model learns patterns from historical operating conditions.

During deployment, incoming measurements are transformed into the same numerical representation used during training.

The network then produces an output indicating whether the current measurements resemble previously observed operating conditions.

Engineers can combine this output with maintenance records, physical inspection, safety procedures, and domain expertise.

The important point is that the mathematics does not operate independently from engineering knowledge. The two complement each other.

Essential Tips

Build linear algebra fundamentals

Focus on:

- Vectors
- Matrices
- Matrix multiplication
- Transposes
- Dot products
- Dimensions
- Eigenvalues and eigenvectors

You do not need to memorize every theorem immediately. Focus first on understanding how these concepts appear in neural networks.

Learn calculus through gradients

For deep learning, concentrate on:

- Functions
- Derivatives
- Partial derivatives
- Chain rule
- Gradients
- Optimization

The chain rule is especially important because backpropagation relies heavily on repeated application of this idea.

Understand probability

Learn:

- Probability distributions
- Conditional probability
- Expected value
- Variance
- Likelihood
- Bayesian concepts

These ideas become useful when interpreting predictions and uncertainty.

Practice with small models

Before studying enormous transformer architectures, experiment with simple neural networks.

A small model makes it easier to observe:

input → prediction → loss → gradient → update

Connect mathematics to engineering

Whenever you learn a mathematical concept, ask:

> *"Where does this appear inside a real neural network?"*

This question turns abstract mathematics into practical knowledge. 🔬

FAQs

What mathematics is required for deep learning?

The most important foundations are linear algebra, calculus, probability, statistics, and optimization. Basic numerical methods are also useful.

Do I need advanced mathematics before learning deep learning?

No. You can begin with basic linear algebra, derivatives, probability, and optimization concepts and progressively develop deeper knowledge.

Why is linear algebra important?

Neural networks process large amounts of numerical information using vectors and matrices. Many fundamental neural-network operations are naturally expressed using linear algebra.

Why is calculus used in neural networks?

Calculus provides the tools needed to determine how changes in model parameters affect the loss. Gradients then provide information used by optimization algorithms.

What is backpropagation?

Backpropagation is a method for efficiently calculating how the network's loss changes with respect to parameters throughout the network.

What is gradient descent?

Gradient descent is an optimization approach that uses gradient information to iteratively adjust parameters toward configurations associated with lower loss.

Is probability necessary for deep learning?

Probability is highly useful for understanding uncertainty, classification outputs, likelihood-based methods, generative models, and statistical behavior.

Can engineers learn deep-learning mathematics without becoming mathematicians?

Yes. Engineers can learn the mathematical concepts most relevant to their applications without mastering every area of pure mathematics.

Conclusion

The mathematics of deep learning is not one isolated subject. It is an interconnected system of ideas.

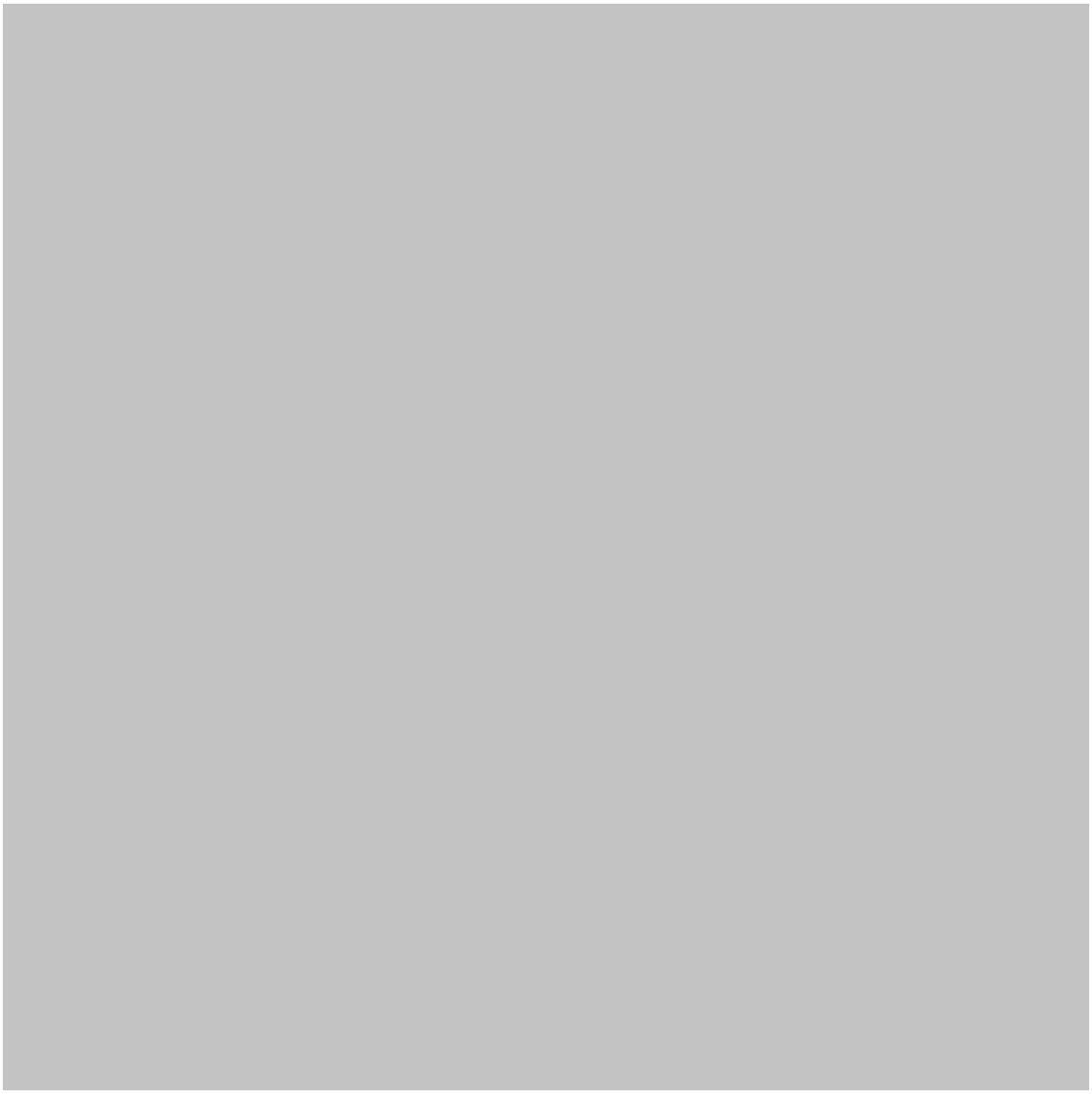
Linear algebra provides the language for representing and transforming data.

Calculus explains how neural-network parameters influence the loss.

Probability and statistics help describe uncertainty and data behavior.

Optimization provides methods for improving model parameters.

Together, these concepts form the mathematical foundation behind modern neural networks.

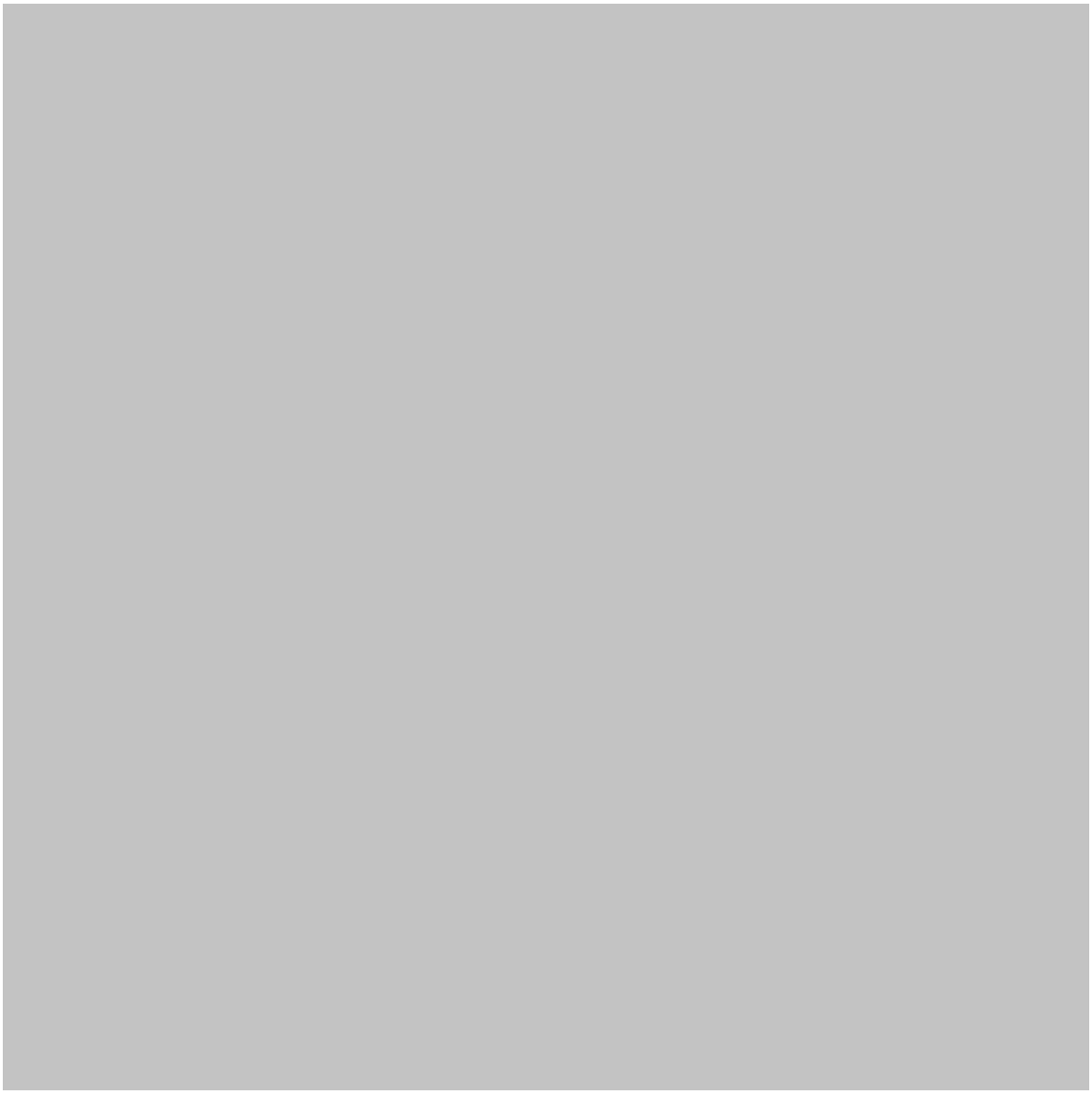


For students, the most effective path is usually progressive: begin with vectors and matrices, move into derivatives and gradients, study probability and statistics, and then connect these ideas to optimization and neural-network architectures.

For professional engineers, the same foundation provides a framework for understanding model behavior rather than simply operating machine-learning libraries.

Ultimately, deep learning can be viewed as a continuous mathematical learning process:

Represent → Transform → Predict → Measure → Differentiate → Optimize → Repeat.



Once this cycle becomes familiar, many concepts that initially appear mysterious—including neural-network training, backpropagation, loss functions, and optimization—become much easier to understand. 🚀

Related Posts:



Statistical Theory 2nd Edition



Introduction to Statistical Decision Theory



Power Pivot and Power BI



Introduction to Modern Statistics



Python Adventures for Young Coders

Preview PDF Book

← Previous Book

Next Book →

EngiVerse

Explore a vast library of free engineering ebooks curated to help you expand your technical knowledge, conduct groundbreaking research, and advance your career. Our mission is to create a hub of knowledge that fosters innovation, empowers aspiring engineers, and bridges the gap between education and industry.

Home Books About Us Contact
Privacy Policy DMCA

copyright 2025 EngiVerse. All Right Reserved